

Programming for Problem Solving

1. Problem solving is a key skill in programming, which allows you to develop software applications that can solve complex problems.
2. Problem solving involves breaking down a problem into smaller, more manageable parts and then using a programming language to create a solution.
3. To solve a problem, you must first identify the problem and then determine the best approach to solve it.
4. Once the problem is identified, you must then decide which programming language to use to create the solution.
5. Common programming languages used for problem solving include Java, C++, Python, and JavaScript.
6. To create a solution, you must first understand the problem and the data associated with it.
7. You must then create a plan that outlines the steps needed to solve the problem.
8. Once the plan is created, you must then write code that implements the plan.
9. Finally, you must test the code to ensure that it produces the desired results.
10. Problem solving is an essential skill for any programmer and can help you create efficient and effective software applications.

Unit 1 - Introduction to Components of a Computer System: Memory, Processor, I/O Devices, Storage, Operating System, Concept of Assembler, Compiler, Interpreter, Loader and Linker

- Memory is the internal storage area in a computer system where data, instructions, and information are held for quick access by the processor.
- Processor is the central unit of a computer system that is responsible for executing instructions and manipulating data.
- I/O devices are hardware components that allow the computer to communicate with the outside world. Examples of I/O devices include keyboards, mice, monitors, printers, and scanners.
- Storage is the component of a computer system that holds data and instructions for future use. Examples of storage devices include hard drives, solid state drives, and optical drives.
- Operating system is the software that controls and manages the computer's hardware and software resources. Examples of operating systems include Windows, macOS, and Linux.
- Assembler is a computer program that translates assembly language into machine language.
- Compiler is a computer program that translates a high-level language into

assembly language or machine language.

- Interpreter is a computer program that executes instructions written in a high-level programming language.
- Loader is a computer program that loads programs into memory for execution.
- Linker is a computer program that links object code modules to form an executable program.

Idea of Algorithm:

- Representation of Algorithm: An algorithm is a set of instructions or rules that are followed in order to solve a problem. Algorithms can be represented in a variety of ways, such as flowcharts, pseudo code, natural language, or even source code.
- Flowchart: A flowchart is a graphical representation of an algorithm, consisting of shapes and arrows that represent the steps of the algorithm.
- Pseudo Code: Pseudo code is a way of representing an algorithm using a combination of natural language and programming language-like syntax. It is often used when developing an algorithm, as it provides a higher-level view of the problem.
- Examples: Examples are an important part of understanding algorithms, as they provide concrete examples of how an algorithm works.
- From Algorithms to Programs: Algorithms can be translated into programs, which are written in a programming language and can be executed by a computer.
- Source Code: Source code is the actual code that is written in a programming language. It is the code that is used to create a program.

Programming Basics: Structure of C Program

- A basic C program consists of one or more functions and a main function, which is the entry point for the program.
- The main function is the first function to be executed when the program starts running.
- The main function is used to call other functions and execute them in order.

Writing and Executing the First C Program

- To write a C program, you will need a text editor and a compiler.
- The text editor is used to write the source code, while the compiler is used to compile the source code and generate an executable file.
- Once the program is written, it can be executed by running the executable file.

Syntax and Logical Errors in Compilation

- Syntax errors are mistakes in the source code that prevent the program from being compiled.
- Logical errors are mistakes in the source code that do not prevent the program from being compiled, but cause the program to behave incorrectly.

Object and Executable Code

- Object code is the compiled version of the source code and is usually written in machine language.
- Executable code is the code that is ready to be executed by the computer, and is usually written in assembly language.

Components of C Language

- Standard I/O in C
 - Standard I/O in C provides functions to read and write data from and to the standard input and output devices such as the keyboard and screen. The most commonly used functions are **printf()** and **scanf()**.
- Fundamental Data Types
 - C language supports a wide range of data types such as **char**, **int**, **float**, **double**, **long**, **short**, **unsigned**, **signed**, etc.
- Variables and Memory Locations
 - Variables are identifiers that are used to store data in memory. Variables are declared with a specific data type and then allocated memory accordingly.
- Storage Classes
 - Storage classes define the scope and lifetime of variables and functions. The storage classes in C language are **auto**, **register**, **static**, and **extern**.

Unit 2 - Arithmetic Expressions and Precedence

Arithmetic expressions are used to represent calculations that use numbers and operators. These expressions can include numeric and relational operators, mixed operands, type conversion, logical operators, bit operations, assignment operator, operator precedence and associativity.

Numeric and Relational Operators

Numeric and relational operators are used to compare two values and return a logical value (true or false). The most common numeric operators are addition (+), subtraction (-), multiplication (*), and division (/). The most common relational operators are greater than (>), less than (<), equal to (==), and not equal to (!=).

Mixed Operands

Mixed operands are used when an expression contains both numeric and relational operators. For example, the expression “ $2 + 3 > 5$ ” contains both a numeric operator (addition) and a relational operator (greater than).

Type Conversion

Type conversion is the process of converting one data type to another. For example, if you have a number stored as a string (e.g. “5”) and you want to perform a calculation on it, you must first convert it to an integer.

Logical Operators

Logical operators are used to combine two or more expressions and return a single logical value (true or false). The most common logical operators are AND (&&), OR (||), and NOT (!).

Bit Operations

Bit operations are used to manipulate individual bits of a number. The most common bit operations are AND (&), OR (|), XOR (^), and NOT (~).

Assignment Operator

The assignment operator (=) is used to assign a value to a variable.

Operator Precedence and Associativity

Operator precedence is the order in which operators are evaluated in an expression. For example, multiplication has a higher precedence than addition, so in the expression “ $2 + 3 * 4$ ”, the multiplication will be evaluated first. Associativity determines the order in which operators with the same precedence are evaluated. For example, addition and subtraction are left-associative, so in the expression “ $2 + 3 - 4$ ”, the addition will be evaluated first.

Conditional Branching: Applying if and Switch Statements, Nesting if and Else and Switch

1. **If Statements** are used to execute code based on a certain condition. An if statement has the following syntax:

```
if (condition) {      // code to be executed }
```

2. An **Else Statement** can be used to execute code when the condition of an if statement is not met. An else statement has the following syntax:

```
if (condition) {      // code to be executed } else {      // code  
to be executed }
```

3. **Nesting If Statements** allows us to check multiple conditions. For example:

```
if (condition1) {      // code to be executed      if (condition2)
{                      // code to be executed      } }
```

4. **Switch Statements** are used to execute code based on multiple conditions. A switch statement has the following syntax:

```
switch (expression) {      case value1:          // code to be
executed                    break;      case value2:          // code to be
executed                    break;      // ...      default:          // code
to be executed }
```

Unit 3 - Iteration and Loops: Use of While, do While and for Loops, Multiple Loop Variables, Use of Break , Goto and Continue Statements

- While loops are used to repeat a block of code until a certain condition is met. The syntax of a while loop is `while (condition) { code block; }`.
- Do-while loops are similar to while loops, except that the code block is executed at least once before the condition is checked. The syntax of a do-while loop is `do { code block; } while (condition)`.
- For loops are used to iterate over a sequence of values. The syntax of a for loop is `for (initialization; condition; increment) { code block; }`.
- Multiple loop variables can be used to control the loop. These variables must be declared before the loop and their values must be updated inside the loop.
- Break statements are used to exit a loop prematurely. The syntax of a break statement is `break;`.
- Goto statements can be used to jump to a specific line of code. The syntax of a goto statement is `goto label;`.
- Continue statements are used to skip the rest of the code in the current loop iteration. The syntax of a continue statement is `continue;`.

Arrays: Array Notation and Representation

- Arrays are data structures that store multiple values of the same data type.
- Array notation is the way to represent an array in code. It is written as a list of values separated by commas and enclosed by brackets.
- The index of an array is the position of the element in the array, and it always starts at 0.

Manipulating Array Elements

- Array elements can be accessed and manipulated using their index.
- Elements can be added, removed, or modified.
- The length of an array can also be changed by adding or removing elements.

Using Multi-Dimensional Arrays

- Multi-dimensional arrays are arrays that contain other arrays.
- They are useful for storing and manipulating data that has multiple dimensions.
- The elements of a multi-dimensional array can be accessed by specifying the indices of each dimension.

Character Arrays and Strings

- Character arrays are arrays of characters (or strings).
- Strings are sequences of characters that are enclosed in double quotes.
- Strings are useful for storing and manipulating text.

Structure, Union, Enumerated Data Types

- Structures, unions, and enumerated data types are user-defined data types.
- Structures are composed of multiple data elements, and unions are composed of multiple data elements of different types.
- Enumerated data types are data types that contain a set of predefined constants.

Array of Structures

- An array of structures is an array that contains multiple structures.
- Each element of the array can be accessed using its index.
- The elements of the structure can then be accessed using the structure's field names.

Passing Arrays to Functions

- Arrays can be passed to functions as arguments.
- The function can then access and manipulate the array elements.
- This is useful for performing operations on multiple elements of an array at once.

Unit 4 - Functions: Introduction

- Functions are a set of instructions that are used to perform a specific task.
- Functions are used to make code more modular and easier to read.

- Functions can take parameters and return values.

Types of Functions

- Built-in functions are functions that are already defined in the language and can be used without any additional code.
- User-defined functions are functions that are defined by the user and can be used in the code.

Functions with Array

- An array is a data structure that stores a collection of values.
- Functions can be used to manipulate arrays.
- Common array functions include sorting, searching and transforming.

Passing Parameters to Functions

- Parameters are values that are passed to a function.
- Parameters can be used to customize the behavior of a function.
- Parameters can be passed by value or by reference.

Call by Value

- Call by value is a method of passing parameters to a function.
- In call by value, the value of the parameter is passed to the function.
- The value of the parameter is not changed in the function.

Call by Reference

- Call by reference is a method of passing parameters to a function.
- In call by reference, the memory address of the parameter is passed to the function.
- The value of the parameter can be changed in the function.

Recursive Functions

- Recursive functions are functions that call themselves.
- Recursive functions can be used to solve complex problems.
- Recursive functions must have a base case to prevent infinite recursion.

Basic of Searching and Sorting Algorithms:

1. Linear Search: Linear search is a simple search algorithm that searches an array, list, or other collection of items for a particular item. It begins at the first element of the collection and compares each element with the target item until it finds a match or reaches the end of the collection.

2. Binary Search: Binary search is an efficient search algorithm that searches a sorted array, list, or other collection of items for a particular item. It begins by comparing the target item to the middle element of the collection. If the target item is not equal to the middle element, the search continues in the appropriate half of the collection, either the upper or lower half.
3. Bubble Sort: Bubble sort is an algorithm that sorts an array of items by repeatedly comparing pairs of adjacent elements and swapping them if they are in the wrong order. The algorithm continues to do this until the array is sorted.
4. Insertion Sort: Insertion sort is an algorithm that sorts an array of items by repeatedly comparing an unsorted item to the items that are already sorted. It begins by comparing the first two items in the array, and then inserts the unsorted item in the correct position.
5. Selection Sort: Selection sort is an algorithm that sorts an array of items by repeatedly finding the smallest item in the unsorted portion of the array and swapping it with the item at the beginning of the unsorted portion. The algorithm then moves to the next unsorted item and repeats the process.

Unit 5 - Pointers: Introduction, Declaration, Applications

- Pointers are variables that store the address of another variable.
- Pointers are declared using the asterisk (*) operator.
- Pointers are used to store the address of a variable, to access the memory location of a variable, to pass arguments to a function by reference and to return multiple values from a function.

Introduction to Dynamic Memory Allocation (Malloc, Calloc, Realloc, Free)

- Dynamic memory allocation is a process of allocating memory at runtime.
- The malloc() function is used to allocate a block of memory of a specified size.
- The calloc() function is used to allocate a block of memory for an array of elements, initialized to zero.
- The realloc() function is used to reallocate memory to a previously allocated block.
- The free() function is used to deallocate a previously allocated memory block.

String and String functions

- A string is an array of characters terminated by a null character ('\0').
- String functions are used to manipulate strings.

- Some of the commonly used string functions are `strcpy()`, `strcat()`, `strlen()`, `strcmp()`, etc.

Use of Pointers in Self-Referential Structures

- Pointers are used to create self-referential structures, such as linked lists and binary trees.
- Self-referential structures are data structures that contain a pointer to another instance of the same data structure.
- This allows for the creation of complex data structures, such as linked lists, binary trees, etc.

Notion of Linked List (No Implementation)

- A linked list is a data structure that consists of a sequence of nodes, each of which contains data and a pointer to the next node in the sequence.
- Linked lists are used to implement dynamic data structures, such as stacks and queues.
- Linked lists can also be used to implement data structures such as trees and graphs.

File Handling: File I/O Functions

File I/O functions are used to perform operations on files such as opening, reading, writing, and closing them. These functions are defined in the `stdio.h` header file.

- **fopen()**: This function is used to open a file. It takes two arguments: the name of the file and the mode in which the file is to be opened (read, write, append, etc.).
- **fclose()**: This function is used to close an open file. It takes one argument: the file pointer.
- **fgetc()**: This function is used to read a single character from a file. It takes one argument: the file pointer.
- **fputc()**: This function is used to write a single character to a file. It takes two arguments: the character to be written and the file pointer.
- **fread()**: This function is used to read a block of data from a file. It takes four arguments: the pointer to the buffer where the data is to be stored, the size of each element, the number of elements, and the file pointer.
- **fwrite()**: This function is used to write a block of data to a file. It takes four arguments: the pointer to the buffer containing the data, the size of each element, the number of elements, and the file pointer.

Standard C Preprocessors

C preprocessors are used to modify the source code before it is compiled. They are invoked by the compiler and can be used to define constants, macros, and

include files.

- **#define**: This directive is used to define constants and macros.
- **#include**: This directive is used to include other source files.
- **#ifdef**: This directive is used to check if a macro is defined.
- **#ifndef**: This directive is used to check if a macro is not defined.
- **#undef**: This directive is used to undefine a macro.

Defining and Calling Macros

Macros are used to define reusable pieces of code that can be called multiple times. They are defined using the **#define** directive.

To define a macro, use the following syntax:

```
#define MACRO_NAME macro_body
```

To call a macro, use the following syntax:

```
MACRO_NAME(arg1, arg2, ...)
```

Command-Line Arguments

Command-line arguments are used to pass parameters to a program when it is executed. They are passed as strings and can be accessed using the **argv** array. The first argument (**argv[0]**) is the name of the program.